

Resposta da atividade parcial

Aluno: Emanuel Rabelo de Souza Sena

Turma:ENGS241M01

1)A característica avaliada é a confiabilidade, especificamente a subcaracterística de Tolerância a Falhas. Pois o teste não tá testando se o desconto é calculado certo, ele tá testando o que acontece quando a gente manda um dado ruim pro método

2)a)São validas: o valor maior que 100 reais que vai ganhar 10% de desconto e valores entre 0 e 100 reais que vão pagar o valor norma.

É invalido: valido menor ou igual a 0 reais

b)O limite de não desconto é R\$100,00 e o desconto é a partir R\$100,01

3)Test case Id: TC-VIP-01

Objetivo do Teste: Verificar se o método calculateDiscount aplica corretamente 20% de desconto para um cliente da categoria VIP quando o valor da compra é positivo

Característica ISO 25010: Adequação Funcional (Functional Suitability) → subcaracterística Correção Funcional

Técnica ISO 29119-4: Partição de Equivalência (Equivalence Partitioning) — testando a classe válida "amount > 0" pra categoria VIP

Entradas (Tier, Amount): ("VIP", 50.00)

Resultado Esperado: Desconto de 20% aplicado → valor final = 40.00

```
4) describe('calculateDiscount - Fronteira categoria PREMIUM', () =>
{
  test('não deve aplicar desconto quando valor for exatamente 100.00', () => {
    const resultado = calculateDiscount('PREMIUM', 100.00);
    expect(resultado).toBeCloseTo(100.00, 2);
  });

  test('deve aplicar 10% de desconto quando valor for 100.01', () =>
  {
    const resultado = calculateDiscount('PREMIUM', 100.01);
    expect(resultado).toBeCloseTo(90.009, 2);
  });
});
```

```
});
```

5) a) Já que a primeira linha da função verifica `typeof amount !== 'number'`, e `'50'` é uma *string*, essa condição é verdadeira, então a função lança imediatamente `Error`, sem sequer chegar na lógica de desconto. Ou seja, diferente do que eu tinha suposto antes, aqui o sistema já está protegido contra esse tipo de entrada.

```
b) test('deve lançar erro ao receber amount como string', () => {  
  expect(() => calculateDiscount('VIP', '50')).toThrow('Valor inválido');  
});
```

6) **Test Design Specification:** É o artefato que descreve o quê vai ser testado e como, antes da execução, as condições de teste, os casos de teste detalhados (entradas, resultados esperados), a lógica de cobertura.

Test Execution Log: registro cronológico do que aconteceu durante a execução (cada teste rodado, quando, resultado, erros/stack traces).

Test Summary Report: um resumo consolidado no final, quantos testes passaram, falharam, cobertura, tempo total, etc.

O relatório do `jest-html-reporters` é um **Test Summary Report**, pois o **design specification** não define o que deveria ser testado.

7 A) O bug foi usar `>=` em vez de `>` na condicional `if (amount >= 100)` ao invés de `if (amount > 100)`, fazendo o valor 100 cair errado na classe "com desconto".

B) Isso viola a **Correção Funcional** porque o sistema não implementa fielmente a regra especificada, todo cliente premium com compra de exatamente R\$ 100,00 recebe um desconto indevido.

8)A) As novas classes que pode ter são: `valor > 500,00 (freeShipping: true)` e `0 < valor ≤ 500,00 (freeShipping: false)`. Fronteiras (BVA): R\$ 500,00 (sem frete grátis) e R\$ 500,01 (com frete grátis).

B) É importante manter os teste automatizados pois garante a **Testabilidade** e **Modificabilidade**, evitando regressões na refatoração.